Web-Technologien UE - Assignment 1

Raphael Bleier, WS 2026

Exercise 1 - HTTP Requests

Gemessen in Chrome (Network-Tab, Cache deaktiviert) am 07.10.2026. Die Zahlen ändern sich bei jedem Neuladen ein bisschen.

a) www.aau.at

Requests	170
Übertragen	ca. 8,2 MB
Ladezeit	DOMContentLoaded nach ca. 2,8 s, Load nach ca. 3,2 s
Status 1. Request	200 OK
Webserver	nginx (steht im Response Header `Server`), dahinter läuft WordPress, sieht man an den `/wp-content/` Pfaden
Ressourcen	1 HTML Dokument, 73 CSS Dateien, 61 JavaScript Dateien, 29 Bilder (png, jpg), 6 Fonts (ttf, woff2)
Langsamster Request	Abgesehen vom Dokument selbst kleine Bilder wie `flags/de.png` mit ca. 2,8 s
Cookies	Bei mir keine, solange man im Cookie-Banner nichts akzeptiert

Was mir aufgefallen ist: Die langsamsten Requests sind nicht die großen Bilder (das Header-Bild hat 1,1 MB), sondern winzige Icons mit 1 KB. Die Seite läuft noch über HTTP/1.1, da macht der Browser nur ca. 6 Verbindungen pro Host auf. Bei 170 Requests müssen die meisten in der Warteschlange warten, und genau das sieht man im Waterfall. Die 73 CSS Dateien sind auch ziemlich viel, das kommt wahrscheinlich von den ganzen WordPress Plugins.

b) www.orf.at

Requests	55
Übertragen	ca. 0,5 MB
Ladezeit	DOMContentLoaded nach ca. 0,9 s, Load nach ca. 1,3 s
Status 1. Request	200 OK (über HTTP/2)
Webserver	Apache
Ressourcen	HTML, 19 JavaScript, 3 CSS, 19 Bilder (jpg, webp, svg, png), 3 Fonts (woff2), dazu 9 XHR/Fetch Requests die JSON nachladen
Langsamster Request	`pipe.orf.at/.../injector.js` mit ca. 0,7 s
Cookies	`didomi_token` (speichert was man im Consent-Banner ausgewählt hat). Dazu kommen Requests an Werbe- und Analytics-Server (adition.com, adnz.co, app-analytics.orf.at), die eigene Drittanbieter-Cookies setzen können

ORF ist trotz viel mehr Inhalt deutlich schneller. Das liegt an HTTP/2 (alle Requests über eine Verbindung) und daran, dass die Bilder klein und teilweise schon als webp ausgeliefert werden. Dafür werden Requests an ca. 12 verschiedene Hosts geschickt, vieles davon ist Tracking und Werbung.

c) Google Suche "Klagenfurt"

Bei jedem Buchstaben, den man eintippt, schickt die Seite einen eigenen GET Request an `/complete/s`. Der Parameter `q` wächst mit: zuerst `q=K`, dann `q=Kl`, `q=Kla` usw. bis `q=Klagenfurt`, also 10 Requests für 10 Buchstaben. `cp` ist die Cursorposition. Die Antwort ist JSON mit Suchvorschlägen, die dann im Dropdown unter dem Suchfeld angezeigt werden (Autocomplete). Für einen Vorschlag wurde auch ein Vorschaubild von `encrypted-tbn0.gstatic.com` geladen. Nebenbei gehen noch POST Requests an `/gen_204` und `play.google.com/log` raus, die loggen nur Klicks und Timing, für die Suche selbst braucht man die nicht. Die ganze Seite wird dabei nie neu geladen, das läuft alles asynchron im Hintergrund.

d) Cache

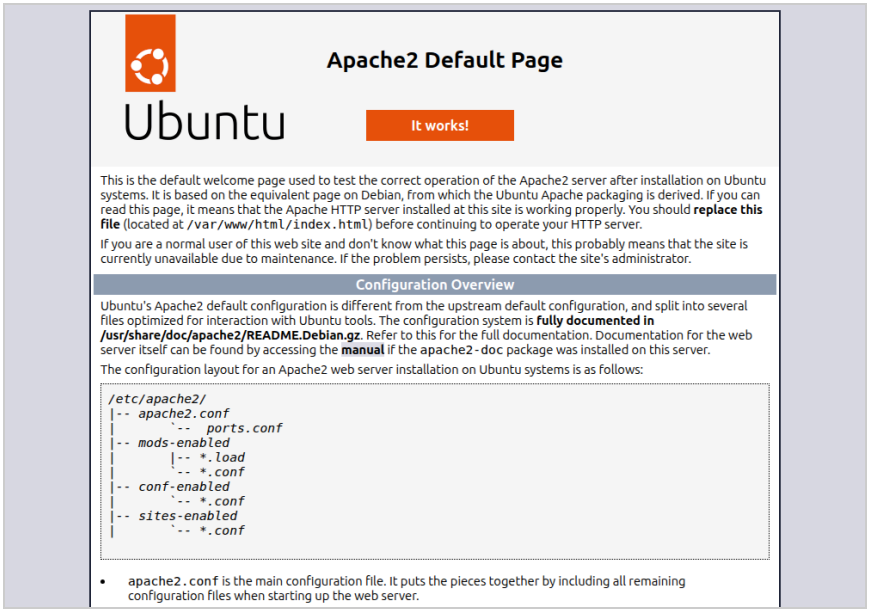
Der Browser-Cache ist ein lokaler Zwischenspeicher. Dort legt der Browser schon geladene Dateien ab (Bilder, CSS, JS, ...). Beim nächsten Besuch nimmt er sie von der Festplatte statt sie nochmal vom Server zu holen, solange sie laut Header (`Cache-Control`, `Expires`, `ETag`) noch gültig sind. Spart Zeit und Datenvolumen.

Löschen geht in Chrome über Einstellungen > Datenschutz > Browserdaten löschen > "Bilder und Dateien im Cache" oder mit Strg+Shift+Entf. In den DevTools kann man im Network-Tab "Disable cache" anhaken, dann wird nichts gecacht solange die DevTools offen sind. Strg+Shift+R lädt eine einzelne Seite ohne Cache neu.

Sinnvoll ist das beim Entwickeln: Wenn ich CSS oder JS ändere und der Browser noch die alte Version im Cache hat, sehe ich meine Änderung nicht. Auch beim Messen wie in Aufgabe a) und b) sollte der Cache aus sein, sonst kommen viel zu kleine Zahlen raus. Und wenn eine Seite kaputt angezeigt wird, weil eine alte Datei im Cache hängt.

Exercise 2 - Apache

Installiert auf Ubuntu mit `sudo apt install apache2` (Apache/2.4.66). Danach zeigt `http://localhost` die Default-Seite mit "It works!":



conf/httpd.conf, ServerRoot, DocumentRoot, Virtual Host

`httpd.conf` ist die Hauptkonfigurationsdatei von Apache. Da steht drin, auf welchem Port der Server lauscht, welche Module geladen werden, wo die Dateien liegen, Zugriffsrechte für Ordner, Logs usw. Unter Ubuntu heißt die Datei `/etc/apache2/apache2.conf` und ist auf mehrere Dateien aufgeteilt (`ports.conf`, `sites-enabled/`, `mods-enabled/`), das Prinzip ist aber gleich.

ServerRoot ist das Basisverzeichnis der Apache Installation, also wo Konfiguration und Module liegen. Relative Pfade in der Config beziehen sich darauf. Bei mir `/etc/apache2`, unter Windows z.B. `c:\programme\apache\Apache2.4`.

DocumentRoot ist der Ordner, aus dem die Webseiten ausgeliefert werden. `http://localhost/test.html` zeigt auf `DocumentRoot/test.html`. Bei mir `/var/www/html` (eingestellt in `sites-enabled/000-default.conf`).

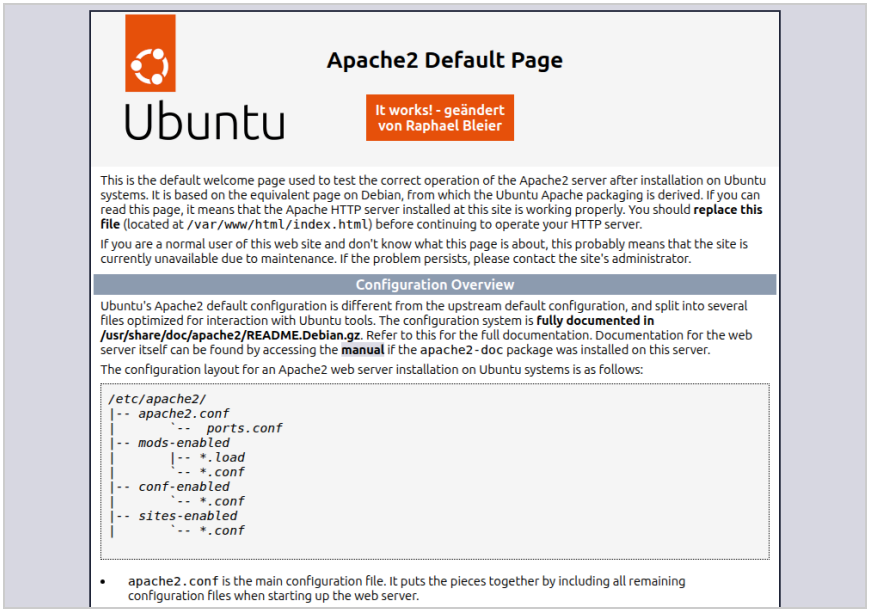
Virtual Host: Damit kann ein einziger Apache mehrere Webseiten hosten. Jeder VirtualHost hat eigene Einstellungen (eigener ServerName, eigene DocumentRoot, eigene Logs) und Apache entscheidet anhand von Hostname, IP oder Port welcher VirtualHost den Request bekommt. Z.B. `seite-a.at` und `seite-b.at` auf derselben Maschine.

127.0.0.1 und localhost

127.0.0.1 ist die Loopback-Adresse. Pakete an diese Adresse verlassen den Rechner nie, sie gehen direkt zurück an den eigenen Computer. `localhost` ist der Hostname dafür (steht in `/etc/hosts`). Verwendet wird das zum Testen: Ich kann meinen Webserver lokal aufrufen ohne Netzwerk und ohne dass er von außen erreichbar sein muss.

index.html und mytest

`index.html` ist die Standarddatei, die Apache ausliefert wenn nur ein Ordner aufgerufen wird (Einstellung `DirectoryIndex`). Deshalb kommt bei `http://localhost` die `/var/www/html/index.html`. Ich hab die Überschrift geändert, nach dem Neuladen sieht man die Änderung sofort:



Dann den Ordner `mytest` mit ein paar Dateien und Unterordnern angelegt. Bei `http://localhost/mytest` kommt eine automatisch generierte Liste aller Dateien und Ordner ("Index of /mytest"):



Das passiert weil es in `mytest` keine index.html gibt und in der Config für `/var/www/` `Options Indexes` aktiv ist. Apache baut dann selbst eine Verzeichnisliste. Auf einem echten Server sollte man das abschalten (`Options -Indexes`), sonst kann jeder sehen welche Dateien rumliegen.

Exercise 3 - HTML Page 1

Siehe `ex3/index.html`, getestet unter `http://localhost/ex3/`.

Exercise 4 - HTML Page 2

Siehe Ordner `ex4`. Die Weiterleitung nach 7 Sekunden (b) ist mit `<meta http-equiv="refresh" content="7; url=...">` gemacht, img3 springt wieder auf img1.

Warum alt?

Das alt-Attribut ist der Ersatztext für ein Bild. Screenreader lesen ihn für blinde oder sehbehinderte Menschen vor, ohne alt wissen die nicht was auf dem Bild ist. Er wird auch angezeigt wenn das Bild nicht geladen werden kann (falscher Pfad, langsame Verbindung). Und Suchmaschinen verstehen

über alt, was auf dem Bild zu sehen ist. Deshalb sollte drinstehen, was man tatsächlich sieht ("Mount Staff seen from hike around Goldeck"), und nicht sowas wie "bild1".

Exercise 5 - HTML Tables

Siehe `ex5/ex5.html`. Kopfzeile in `<thead>` mit `scope="col"`, die Unis als `<th scope="row">` im `<tbody>`. Damit greifen die vorhandenen CSS Regeln automatisch (dunkle Kopfzeile, jede zweite Zeile grau, Uni-Namen links). Die Caption mit dem Link landet durch `caption-side: bottom` unter der Tabelle.

Exercise 6 - Forms

Siehe `ex6/ex6.html`. Validierung nur mit HTML: `required`, `type="email"`, `pattern` beim Telefon, `min/max` bei den Passagieren, `maxlength="512"` beim Kommentar. Jedes Feld hat ein `name`, das ist in der Mail dann der Key, also z.B. `email=max@edu.aau.at`.

Exercise 7 - CSS Selectors

Wichtig zum Verständnis: `<p>` in `<h1>`/`<h2>` ist eigentlich kein gültiges HTML, der Browser lässt die p aber trotzdem im h1/h2 drin. Rot heißt hier: der Text wird rot angezeigt.

Regel	**Ergebnis**
`#content, p`	Komma = zwei getrennte Selektoren. Das div #content wird rot (vererbt sich auf alles drin) und zusätzlich jedes p auf der Seite. Damit ist alles rot außer die erste h2, "A heading" und der Link.
`#content p`	Alle p irgendwo innerhalb von #content (Nachfahren, egal wie tief). Also paragraph 1 und 2, die 4 p im h2 und die 2 p im h1.
`#content h2 p`	Nur p, die in einem h2 in #content liegen: "paragraph 1-4 in h2".
`#content > p`	`>` heißt direktes Kind. Nur "paragraph 1" und "paragraph 2", die im h1/h2 sind Enkel und zählen nicht.
`#content h1 p`	Die zwei p im h1 von #content: "Text within h1 and p" und "... 2".
`div p:nth-child(2)`	p in einem div, die das 2. Kind ihres Elternelements sind: "paragraph 2", "paragraph 2 in h2", "h1 and p 2" und in .othercontent "p1" (weil dort das h1 das 1. Kind ist).
`div p:nth-child(2n)`	Gleich, aber jedes gerade Kind (2, 4, 6...): zusätzlich "paragraph 4 in h2" und "p3".
`h2`	Alle h2 werden rot: "This is some text" und durch Vererbung auch alle p, die in einem h2 stecken.
`div.othercontent > p:first-child`	Nichts. Das erste Kind von .othercontent ist das h1, kein p, also passt kein Element.
`div.othercontent h1 + p`	`+` = das direkt folgende Geschwister. Nur "p1", weil es direkt nach dem h1 kommt.
`div.othercontent h1 ~ p`	`~` = alle folgenden Geschwister vom Typ p: "p1" bis "p4".
`a[title^=other]:link`	Links, deren title mit "other" anfängt und die noch nicht besucht wurden: "Some other Link".

`body a[title^=other]:link`	Gleiches Ergebnis, das body davor schränkt nichts ein, weil eh jeder Link im body ist. Nur die Spezifität ist etwas höher.
`.othercontent * p`	p, die mindestens eine Ebene tiefer in .othercontent liegen (irgendein Element dazwischen). Also nur die p im h2 und im h1, nicht p1-p4, die sind direkte Kinder.

Exercise 8 und 9 - CSS Layout / Responsive

Siehe `ex8/ex8.html` und `ex9/ex9.html`, nur der style-Block wurde befüllt. Kurz zur Umsetzung: Body ist eine Flexbox-Spalte mit `min-height: 100vh` und der Content hat `flex: 1`, dadurch ist der Footer immer unten. Header und Menü sind auch Flexboxen. Bilder floaten links, mit `section:nth-of-type(even)` rechts. "(external)" kommt über `a[href^="http"]::after`. Bei ex9 kommt eine Media Query für `max-width: 600px` dazu: Menü untereinander als graue runde Buttons, Bilder ausgeblendet, Footer-Links als Buttons.